ПРОЕКТ

по Структури от данни

Тема: Електронна таблица

Изготвил: Ивайло Юлиянов Макаков

Факултетен номер: 5MI0700244

Специалност: Информационни системи

Курс: Трети

Преподавател: доц. Армянов

СЪДЪРЖАНИЕ

Въведение 3
Архитектура и дизайн 4
Йерархия на клетките 4
Йерархия на изразите 6
Използвани структури от данни 7
Разредена матрица 7
Абстрактно Синтактично Дърво 7
Описание на Алгоритмите 8
Парсер 8
Алгоритми 8
Визуализация 8
Управление на паметта 9
Файлови формат 9
Заключение 9

Въведение

Настоящият проект представлява софтуерна реализация на електронна таблица с команден интерфейс (CLI), разработена на езика C++. Системата е проектирана да обработва и манипулира данни, организирани в двумерна мрежа от клетки, като поддържа динамично изчисляване на математически и логически изрази.

Основната цел на проекта е да предостави функционалност, близка до тази на комерсиалните продукти (като Excel), но в олекотен формат. Системата позволява въвеждане на различни типове данни (числа, текст, формули), поддържа релативно и абсолютно адресиране на клетки (напр. R1C1 и R[-1]C[0]), както и агрегиращи функции върху диапазони от данни (SUM, AVG, IF).

Ключова характеристика на реализацията е способността ѝ да работи с теоретично неограничени размери на таблицата (напр. до индекс INT_MAX), благодарение на използването на разредена матрица (Sparse Matrix). Това позволява ефективно използване на ресурсите, като се заделя памет само за клетките, които реално съдържат информация.

Архитектура и дизайн

Системата е реализирана като модулна обектно-ориентирана структура, при която всяка основна функционалност е отделена в самостоятелен компонент. Основната идея е логиката за съхранение на данни, логиката за изчисление и логиката за парсване да бъдат ясно разграничени. Използвани са подходящи Структури от данни.

Архитектурата следва принципите на:

- Абстракция – чрез общ интерфейс за клетките;
- Полиморфизъм – чрез виртуални методи и динамично свързване;
- Разделение на отговорностите (Separation of Concerns) – отделяне на логиката за съхранение, парсване и изчисление;
- Безопасно управление на динамична памет – чрез имплементация на Rule of 6.

Системата е изградена от три основни логически компонента:

- Модел на таблицата (Spreadsheet) - управлява таблицата и всички операции върху нея
- Модел на клетките (Cell и наследници)
- Парсер и AST (абстрактно синтактично дърво) - представят формулите като дървовидна структура

Тази организация позволява всяка част от системата да има ясно определена отговорност и улеснява разширяването на функционалността.

Йерархия на клетките

Базовият абстрактен клас Cell дефинира общ интерфейс за всички типове клетки. Той съдържа виртуални методи за:

- получаване на стойност (getValue)
- текстово представяне (toString)
- клониране (clone)

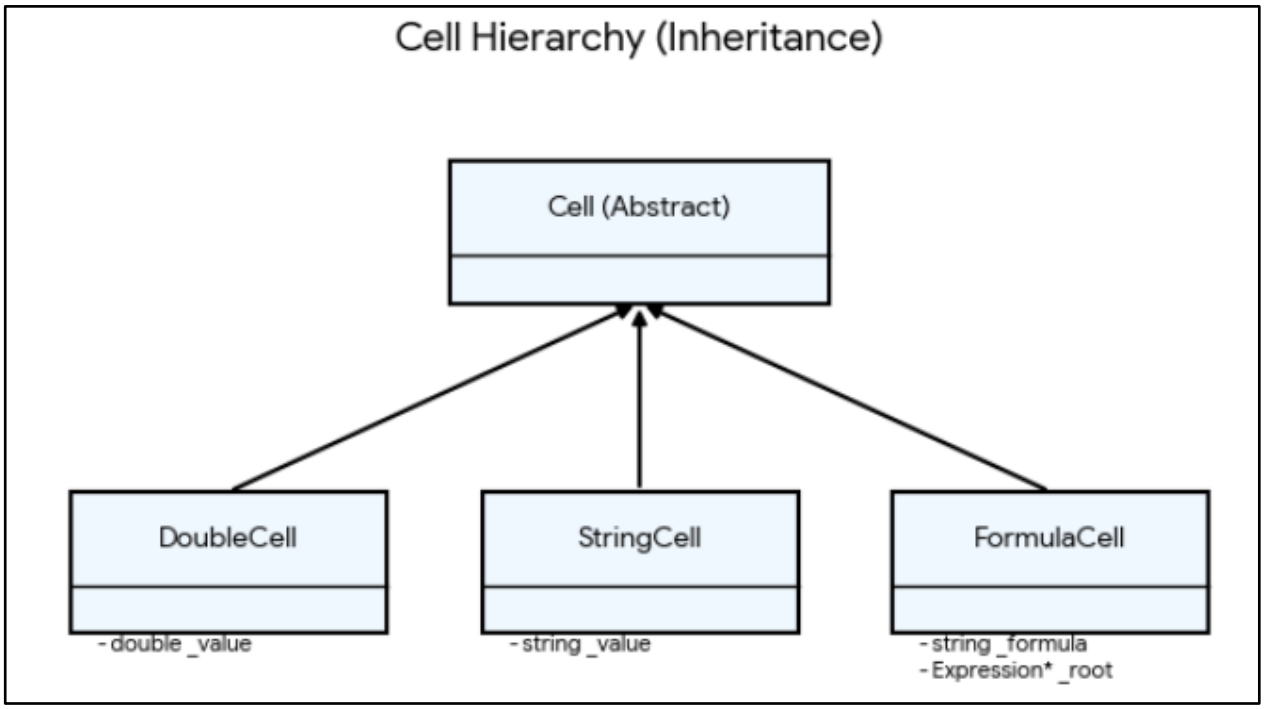
Използването на полиморфизъм позволява различните типове клетки да се съхраняват чрез указатели към базовия клас, но да се държат според конкретния си тип.

От Cell наследяват три класа:

- DoubleCell – съхранява числова стойност от тип double.
- StringCell – съхранява текстова стойност. При нужда може да бъде преобразувана в число.

- FormulaCell – съхранява формула. При създаване формулата се парсва и се изгражда Абстрактно Синтактично Дърво (AST), което се използва при всяко преизчисляване.

Това решение избягва повторно парсване на текста и прави изчисленията по-ефективни.



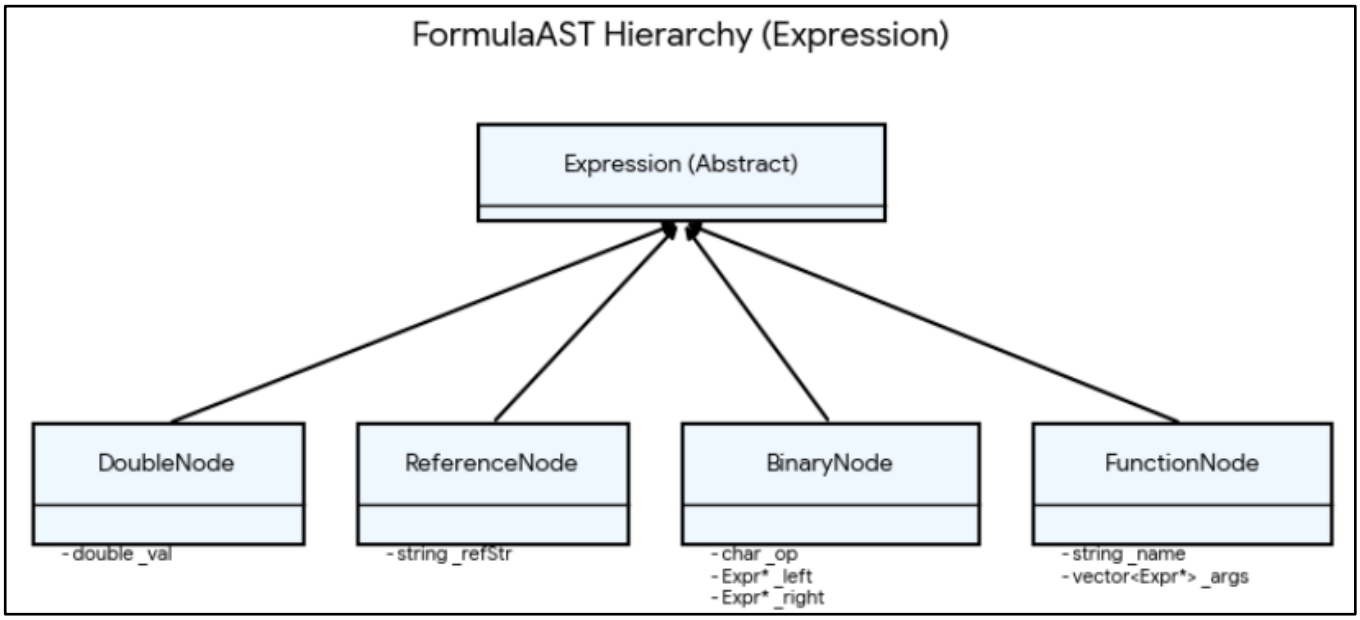
Йерархия на изразите

Формулите се представят като дървовидни структури. Базовият клас Expression има чисто виртуален метод eval(...).

- DoubleNode: Листо в дървото, представляващо константа.
- ReferenceNode: Листо, представляващо референция към друга клетка (абсолютна или релативна).
- BinaryNode: Вътрешен възел за бинарни операции (+, -, *, /, ==, !=, и др.).
- FunctionNode: Възел за функции с аргументи (SUM, IF, MIN, MAX).

Всеки възел от дървото отговаря само за собствената си логика и при извикване на eval рекурсивно изчислява поддървото си.

Тази структура гарантира правилно спазване на приоритета на операциите и улеснява добавянето на нови функции.



Използвани структури от данни

Разредена матрица

За съхранение на таблицата се използва структурата:

std::map<int, std::map<int, Cell*>> table;

Обосновка: Изискването на задачата допуска адреси от рода на R100000C100000. Използването на двумерен масив би изисквало огромно количество памет за празни клетки. Чрез вложени std::map, памет се заделя само за клетките, които имат стойност.

Избор на числов тип: Въпреки че в условието се споменават цели числа, проектът използва тип double за всички стойности. Това решение е взето с цел коректна работа на делението и функцията AVG (средно аритметично), които иначе биха загубили точност. Операторът за модул (%) е реализиран чрез временно преобразуване към int.

Защита от цикли: Реализиран е алгоритъм за засичане на циклични зависимости (напр. клетка А зависи от Б, а Б от А). При изчисление на формула, текущата клетка се маркира в множеството calculating. Ако алгоритъмът достигне до клетка, която вече е маркирана, се хвърля изключение, предотвратявайки забиване на програмата (Stack Overflow).

Абстрактно Синтактично Дърво

Формулите се представят чрез Абстрактно Синтактично Дърво (AST).

При въвеждане на формула тя не се съхранява само като текст. Вместо това се парсва и се превръща в дърво от обекти в динамичната памет.

Предимства на този подход:

- формулата се парсва само веднъж
- при повторно изчисление не се налага нов синтактичен анализ
- спазва се правилният приоритет на операциите
- лесно могат да се добавят нови операции и функции

Всеки възел в дървото отговаря за собствената си част от израза и при извикване на eval рекурсивно изчислява поддървото си.

Описание на Алгоритмите

Парсер

За превръщането на текстовия низ (напр. =5+2 * R1C1) в дърво се използва алгоритъмът "Рекурсивен спускащ се парсер". Той разделя граматиката на нива за спазване на приоритета на операциите:

1. parseExpression: Обработва най-ниския приоритет (събиране, изваждане, сравнение).
2. parseTerm: Обработва среден приоритет (умножение, деление).
3. parseFactor: Обработва най-висок приоритет (скоби, функции, числа, адреси).

Всяко ниво извиква следващото, което гарантира, че 2 + 3 * 4 се интерпретира като 2 + (3 * 4).

Алгоритми

Методът eval(...) приема текущите координати на клетката (ctxRow, ctxCol).

ReferenceNode използва тези координати, за да изчисли реалния адрес:

- R5C5 → връща стойността от ред 5, колона 5
- R[-1]C[0] → изчислява относителен адрес спрямо текущата клетка

Оценяването на изразите се извършва рекурсивно. Всеки възел извиква eval върху своите наследници и комбинира резултатите според операцията си.

За функцията IF е реализирано мързеливо изчисление – изчислява се само клона, който отговаря на условието.

Визуализация

При команда PRINT VAL ALL се извеждат всички използвани клетки.

Ако таблицата е по-широка от стандартната ширина на терминала (80 символа), тя се разделя на вертикални части.

Алгоритъмът:

- изчислява ширината на всяка колона
- групира колоните така, че да се поберат в 80 символа
- отпечатва ги последователно

Това позволява четимо извеждане дори при широки таблици.

Управление на паметта

Използвани са Rule of 6, Strong Exception Guarantee, виртуални функции.

Файлови формат

Таблицата може да бъде записвана и зареждана чрез командите SAVE и LOAD.

Файлът съдържа информация само за клетките, които имат стойност.

Всяка клетка се записва с нейните координати и суровото ѝ съдържание (число, текст или формула).

За да се спази стриктно изискването на заданието, форматът на файла е стандартен CSV (Comma-Separated Values), където всеки ред от таблицата отговаря на ред във файла, а клетките са разделени с точка и запетая (;).

Оптимизация: Тъй като таблицата може да има огромни индекси (напр. R100000C1), пълният запис би генерирал огромни файлове. Затова алгоритъмът за запис (SAVE) интелигентно определя активната област (Bounding Box) от minRow до maxRow и записва само този диапазон. Това осигурява баланс между съвместимост (валиден CSV) и ефективност.

При зареждане:

- файлът се прочита ред по ред
- за всяка записана клетка се създава съответен обект
- при формула се извършва парсване и изграждане на AST

Този подход гарантира, че след LOAD таблицата е в същото състояние, в което е била при SAVE.

Заключение

Разработената система за електронни таблици представлява пълнофункционално решение, което покрива всички поставени изисквания. Чрез използването на разредена матрица, тя е високоефективна по отношение на паметта.

Чрез внедряването на Абстрактно Синтактично Дърво, тя осигурява бързодействие при изчисленията.

Кодът е написан според най-добрите практики на съвременния C++, с фокус върху безопасността на паметта (no memory leaks) и устойчивостта на грешки (exception safety), което прави проекта стабилен и лесен за бъдещо надграждане.